ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA ĐIỆN TỬ VÀ TRÍ TUỆ NHÂN TẠO



BÁO CÁO ĐỒ ÁN HỌC PHẦN – NHÓM 7

PBL3: MẠNG MÁY TÍNH

ĐỀ TÀI

PGP / OPENPGP

NGHIÊN CỨU VÀ MÔ PHỎNG CƠ CHẾ MÃ HÓA LAI (HYBRID ENCRYPTION) KẾT HỢP RSA VÀ AES-GCM

Giảng viên hướng dẫn: TS. Trần Thị Minh Hạnh

Sinh viên thực hiện (Nhóm 7):

STT	Họ và tên	MSSV
1	Vũ Duy Dương	106230007
2	Huỳnh Thiện Lộc	106230052
3	Phan Vĩnh Sơn	106230059

Đà Nẵng, năm 2026

MỤC LỤC

DANH MỤC TỪ VIẾT TẮT VÀ THUẬT NGỮ **2**

1. Giới thiệu **4**
1.1. Bối cảnh và phân biệt PGP với OpenPGP 4
1.2. Mục tiêu và phạm vi đề tài 5
1.3. Phân công nhiệm vụ của các thành viên 6

2. Các nghiên cứu và phương pháp liên quan **6**
2.1. Tổng quan mã hóa đối xứng AES-GCM và bất đối xứng RSA 6
2.2. Chuẩn OpenPGP và lý do kết hợp mã hóa lai 7

3. Phương pháp thực hiện **8**
3.1. Quy trình mã hóa lai và phân vai thuật toán 8
3.2. Cơ sở toán học và kích thước khóa của RSA 10
3.3. Lý thuyết và lưu đồ thuật toán mã hóa khối AES-128 11
3.4. Chế độ mã hóa xác thực AES-GCM (AEAD) 13
3.5. Đóng gói khung truyền thông Socket TCP và xác thực dấu vân tay 14

4. Thực nghiệm và kết quả **15**
4.1. Ví dụ số học minh họa thuật toán RSA trên không gian nhỏ 15
4.2. Kết quả kiểm thử ba kịch bản an toàn 17
4.3. Thực nghiệm truyền thông Socket TCP giữa hai máy mạng LAN 18
4.4. Giao diện mô phỏng trực quan trên nền web 19

5. Kết luận và hướng phát triển **20**

TÀI LIỆU THAM KHẢO **21**

PHỤ LỤC **22**
Phụ lục A: Cấu trúc mã nguồn và hướng dẫn chạy chương trình 22
Phụ lục B: Cài đặt giải thuật Euclid mở rộng và RSA số học 22
Phụ lục C: Cài đặt AES-GCM và bộ đóng gói mã hóa lai 24
Phụ lục D: Cài đặt truyền thông Socket TCP giữa 2 máy mạng LAN 25

DANH MỤC TỪ VIẾT TẮT VÀ THUẬT NGỮ

Bảng 1. Danh mục các từ viết tắt và thuật ngữ sử dụng trong báo cáo

Từ viết tắt	Tên tiếng Anh đầy đủ	Giải nghĩa tiếng Việt
AEAD	Authenticated Encryption with Associated Data	Mã hóa xác thực có dữ liệu bổ sung
AES	Advanced Encryption Standard	Tiêu chuẩn mã hóa nâng cao
CA	Certificate Authority	Cơ quan cấp phát chứng chỉ số
CBC	Cipher Block Chaining	Chế độ nối khối bản mã
CRT	Chinese Remainder Theorem	Định lý số dư Trung Hoa
CSPRNG	Cryptographically Secure Pseudorandom Number Generator	Bộ sinh số giả ngẫu nhiên an toàn mật mã
CTR	Counter Mode	Chế độ bộ đếm
ECB	Electronic Codebook	Chế độ sổ mã điện tử
GCM	Galois/Counter Mode	Chế độ Galois kết hợp bộ đếm
GF	Galois Field	Trường hữu hạn Galois
GHASH	Galois Hash Function	Hàm băm đại số trên trường hữu hạn
IV / Nonce	Initialization Vector / Number Used Once	Vector khởi tạo / Giá trị dùng một lần
JSON	JavaScript Object Notation	Định dạng trao đổi dữ liệu văn bản có cấu trúc
LAN	Local Area Network	Mạng cục bộ
MAC	Message Authentication Code	Mã xác thực thông điệp
MITM	Man-in-the-Middle	Tấn công kẻ đứng giữa
NIST	National Institute of Standards and Technology	Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ
OAEP	Optimal Asymmetric Encryption Padding	Cơ chế đệm mã hóa bất đối xứng tối ưu

OpenPGP	Open Pretty Good Privacy	Chuẩn giao thức mã hóa thư tín mở của IETF
PGP	Pretty Good Privacy	Phần mềm và mô hình mã hóa lai do Phil Zimmermann phát triển
RFC	Request for Comments	Tài liệu đặc tả kỹ thuật của IETF
RSA	Rivest–Shamir–Adleman	Hệ mật mã khóa công khai bất đối xứng RSA
SHA-256	Secure Hash Algorithm 256-bit	Hàm băm an toàn tạo chuỗi băm 256-bit
SPN	Substitution–Permutation Network	Mạng thay thế – hoán vị
TCP	Transmission Control Protocol	Giao thức điều khiển truyền vận
XOR	Exclusive OR	Phép toán cộng loại trừ theo từng bit

NGHIÊN CỨU VÀ MÔ PHỎNG CƠ CHẾ MÃ HÓA LAI (HYBRID ENCRYPTION) KẾT HỢP RSA VÀ AES-GCM THEO CHUẨN PGP/ OPENPGP

Vũ Duy Dương (106230007), Huỳnh Thiện Lộc (106230052), Phan Vĩnh Sơn (106230059)
duong22012005@gmail.com, locht.bk@gmail.com, sonpv.bk@gmail.com
Khoa Điện tử và Trí tuệ Nhân tạo – Trường Đại học Bách Khoa, Đại học Đà Nẵng

Tóm tắt:

Trong bối cảnh truyền thông mạng mở, việc bảo đảm đồng thời tính bí mật, tính toàn vẹn và tính xác thực dữ liệu đặt ra nhiều yêu cầu kỹ thuật. Mật mã đối xứng AES xử lý nhanh các khối dữ liệu lớn nhưng gặp khó khăn khi phân phối khóa bí mật qua kênh không an toàn. Ngược lại, hệ mật bất đối xứng RSA hỗ trợ trao đổi khóa qua cặp khóa công khai và bí mật nhưng có chi phí lũy thừa mô-đun lớn, không phù hợp để mã hóa trực tiếp thông điệp dài.

Báo cáo trình bày việc nghiên cứu và cài đặt kiến trúc Mã hóa lai (Hybrid Encryption) theo chuẩn PGP/OpenPGP (RFC 4880, RFC 9580). Hệ thống dùng AES-128-GCM để mã hóa dữ liệu và kiểm tra toàn vẹn bằng hàm băm GHASH, kết hợp RSA số học (modulus 256-bit hoặc 512-bit, kiểm tra nguyên tố Miller–Rabin) để bảo vệ khóa phiên dùng một lần (Session Key). Nhóm đã xây dựng mô hình truyền thông Socket TCP giữa Client (Alice) và Server (Bob) có đóng khung chống phân mảnh luồng byte, cùng ứng dụng mô phỏng tương tác trên NiceGUI. Kết quả qua 3 kịch bản kiểm thử cho thấy hệ thống khôi phục chính xác dữ liệu gốc và phát hiện, từ chối các gói tin bị sửa bit trên đường truyền.

Từ khóa: PGP, OpenPGP, RSA, AES-GCM, Mã hóa lai (Hybrid Encryption), Socket TCP, Xác thực toàn vẹn, GHASH, Miller–Rabin, Euclid mở rộng.

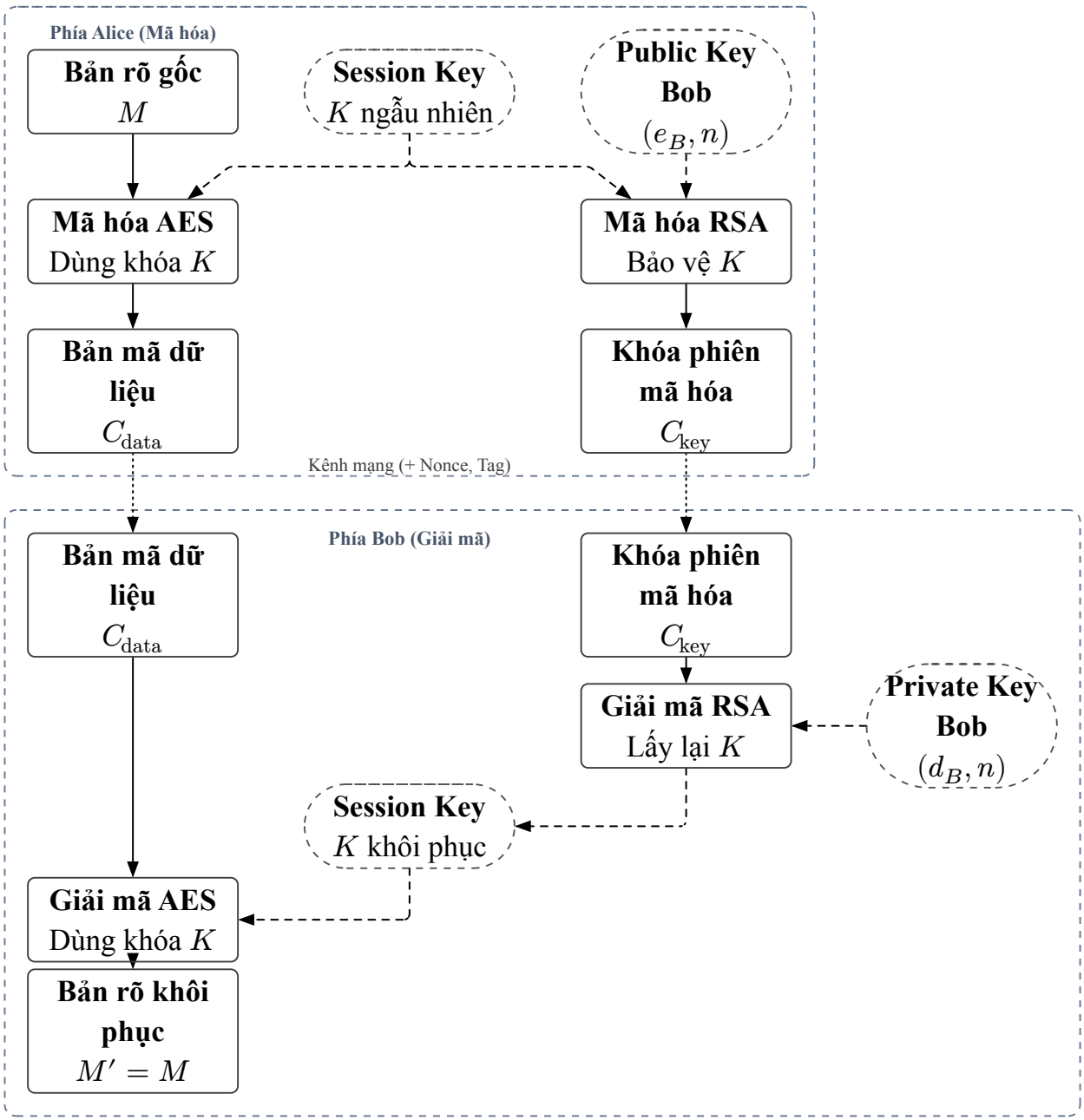
1. Giới thiệu

1.1. Bối cảnh và phân biệt PGP với OpenPGP

Theo tài liệu an toàn mạng [1], truyền thông tin cậy cần bảo đảm bốn tính chất: bí mật, toàn vẹn, xác thực và chống chối bỏ. Để đáp ứng đồng thời hiệu năng và an toàn trao đổi khóa, các hệ thống thực tế kết hợp hai nhánh mật mã: dùng mã hóa đối xứng AES (**Advanced Encryption Standard**, tiêu chuẩn mã hóa nâng cao) [2] cho phần dữ liệu và dùng mã hóa bất đối xứng RSA (**Rivest–Shamir–Adleman**) [3] để bảo vệ khóa trao đổi qua kênh công khai.

Trong thực tế, hai khái niệm PGP (**Pretty Good Privacy**) và OpenPGP thường bị gọi chung nhưng có bản chất khác nhau: PGP là phần mềm bảo mật thư điện tử do Phil Zimmermann phát triển năm 1991, còn OpenPGP là bộ chuẩn giao thức mở do IETF (**Internet Engineering Task Force**) ban hành dựa trên PGP qua các đặc tả RFC 4880 [4] và RFC 9580 [5]. Đề tài này **mô phỏng kiến trúc mã hóa lai theo nguyên lý của**

chuẩn OpenPGP: mã hóa thông điệp bằng khóa phiên AES dùng một lần, rồi mã hóa khóa phiên đó bằng khóa công khai RSA của người nhận như trong Hình 1.



Hình 1. Sơ đồ nguyên lý mã hóa và giải mã lai RSA – AES khép kín trong mô hình PGP [6]

1.2. Mục tiêu và phạm vi đề tài

Đề tài hướng tới các mục tiêu chính trong học phần PBL3: Mạng máy tính:

1. Nắm vững cơ sở toán học của RSA (Euclid mở rộng, định lý Euler, kiểm tra nguyên tố Miller–Rabin) và chế độ xác thực AES-GCM (**Galois/Counter Mode**).
2. Tự cài đặt RSA bằng Python thuần để quan sát rõ từng bước sinh khóa và tính toán số học.
3. Kết nối RSA với AES-GCM thành luồng mã hóa lai hoàn chỉnh: sinh khóa phiên, đóng gói bản tin, giải mã và kiểm tra toàn vẹn.

4. Xây dựng mô hình Client–Server qua Socket TCP (**Transmission Control Protocol**) có đóng khung độ dài để truyền nhận ổn định trong mạng cục bộ (LAN).
5. Phát triển giao diện mô phỏng trực quan và kiểm thử các kịch bản tấn công sửa đổi dữ liệu hoặc dùng sai khóa.

Về phạm vi, hệ thống tập trung bảo đảm tính bí mật, tính toàn vẹn dữ liệu và xác thực thủ công khóa công khai qua dấu vân tay SHA-256. Nhóm dùng cấu hình RSA-256 cho bản demo thuật toán để dễ quan sát số học và RSA-512 cho mô hình Socket LAN nhằm tăng biên độ an toàn mà vẫn sinh khóa nhanh (chi tiết kích thước khóa trình bày ở Mục 3.2). Các thành phần mở rộng của OpenPGP như đệm RSA-OAEP [7] (chống tấn công Coppersmith [8]), chữ ký số, Web of Trust và gói tin nhị phân đa tầng chưa nằm trong phạm vi phiên bản này.

1.3. Phân công nhiệm vụ của các thành viên

Công việc nghiên cứu lý thuyết, cài đặt thuật toán, lập trình mạng và kiểm thử được phân chia cho ba thành viên như trong Bảng 2.

Bảng 2. Phân công nhiệm vụ của nhóm

Tên thành viên	Nhiệm vụ	Phần trăm đóng góp
Vũ Duy Dương (`106230007`)	Trưởng nhóm; thiết kế kiến trúc mã hóa lai; cài đặt RSA số học thuần (Miller–Rabin, Euclid mở rộng); phát triển giao diện mô phỏng trực quan; tổng hợp báo cáo.	34%
Huỳnh Thiện Lộc (`106230052`)	Nghiên cứu cơ chế mã hóa xác thực AES-GCM và hàm băm GHASH trên trường hữu hạn Galois; thiết kế và thực thi các kịch bản kiểm thử toàn vẹn.	33%
Phan Vĩnh Sơn (`106230059`)	Nghiên cứu giao thức Socket TCP; xây dựng mô hình Client–Server truyền nhận qua mạng LAN; cài đặt đóng khung tiền tố độ dài và đối chiếu dấu vân tay SHA-256.	33%

2. Các nghiên cứu và phương pháp liên quan

2.1. Tổng quan mã hóa đối xứng AES-GCM và bất đối xứng RSA

Năm 2001, Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ (NIST) ban hành Tiêu chuẩn Mã hóa Nâng cao AES theo FIPS PUB 197 [2] nhằm thay thế chuẩn DES đã lỗi thời. AES là hệ mật mã khối có kích thước khối cố định 128 bit, hỗ trợ khóa 128, 192 hoặc 256 bit, vận hành theo cấu trúc mạng thay thế – hoán vị (SPN). Trong

các chế độ hoạt động truyền thống như ECB (**Electronic Codebook**) hay CBC (**Cipher Block Chaining**), hệ mật chỉ bảo vệ tính bí mật mà không tự phát hiện được việc sửa đổi bản mã trên đường truyền. Để khắc phục hạn chế này, McGrew và Viega đề xuất chế độ GCM (**Galois/Counter Mode**) năm 2004 [9] và được NIST chuẩn hóa trong SP 800-38D [10]. AES-GCM thuộc nhóm mật mã xác thực có dữ liệu bổ sung AEAD (**Authenticated Encryption with Associated Data**), kết hợp mã hóa dòng theo bộ đếm (CTR) với hàm băm đại số GHASH trên trường hữu hạn $\mathrm{GF}(2^{128})$ để vừa giữ bí mật nội dung vừa sinh thẻ xác thực 128-bit chống giả mạo.

Về nhánh mật mã khóa công khai, khái niệm trao đổi khóa không cần chia sẻ bí mật trước do Diffie và Hellman đề xuất năm 1976 [11] là nền tảng cho sự ra đời của thuật toán RSA năm 1977 [3]. Độ an toàn của RSA dựa trên tính khó của bài toán phân tích một hợp số nguyên lớn $n = p \cdot q$ ra hai thừa số nguyên tố p, q. Chuẩn PKCS 1 / RFC 8017 [7] quy định cặp khóa gồm khóa công khai (e, n) dùng để mã hóa và khóa bí mật (d, n) dùng để giải mã, đồng thời hỗ trợ chiều ngược lại cho chữ ký số.

2.2. Chuẩn OpenPGP và lý do kết hợp mã hóa lai

Chuẩn OpenPGP (RFC 4880 [4], RFC 9580 [5]) không định nghĩa thuật toán mật mã mới mà đóng vai trò kiến trúc tích hợp các giải thuật tiêu chuẩn: dùng mã hóa đối xứng để bảo vệ nội dung thông điệp, dùng mã hóa bất đối xứng để bảo vệ khóa phiên và dùng hàm băm kết hợp chữ ký số để xác thực danh tính. Sự khác biệt kỹ thuật giữa AES-128-GCM và RSA được tổng hợp trong Bảng 3.

Bảng 3. So sánh đặc tính kỹ thuật giữa AES-128-GCM và RSA

Tiêu chí so sánh	AES-128-GCM (Đối xứng)	RSA-256 / 512 (Bất đối xứng)
Số lượng khóa	Dùng chung 1 khóa phiên bí mật K (128-bit) cho cả hai đầu.	Dùng cặp khóa công khai (e, n) và khóa bí mật (d, n).
Tốc độ xử lý	Nhanh nhờ các phép toán dịch byte, tra bảng và XOR.	Chậm do tính lũy thừa mô-đun trên số nguyên lớn.
Kích thước dữ liệu	Phù hợp với thông điệp và tệp tin dung lượng lớn.	Chỉ mã hóa trực tiếp giá trị số học nhỏ hơn modulus n.
Phân phối khóa	Khó khăn, cần kênh truyền an toàn để chia sẻ khóa K.	Thuận tiện, khóa công khai phát tán tự do qua mạng mở.
Kiểm tra toàn vẹn	Tích hợp sẵn thẻ xác thực 128-bit từ hàm băm GHASH.	Không tự kiểm tra toàn vẹn; cần chữ ký số hoặc dùng thêm đệm RSA-OAEP để bảo mật IND-CCA2.

Do hai hệ mật bù trừ trực tiếp cho nhau, giải pháp mã hóa lai được hình thành theo nguyên tắc phân vai rõ ràng: AES đảm nhận mã hóa toàn bộ dữ liệu bản rõ M bằng khóa phiên ngẫu nhiên K, còn RSA đảm nhận mã hóa chính khóa phiên K bằng khóa công khai e_B của người nhận. Công thức đóng gói tổng quát của một gói tin mã hóa lai được biểu diễn ở Phương trình 1:

$$\text{PGP_Packet} = \left\{ \text{RSA}_{e_B}(K), \quad \text{AES}_K(M) \right\} \tag{1}$$

Trong mô hình này, kích thước khóa RSA 256-bit hoặc 512-bit được dùng cho mục đích học thuật và mô phỏng trực quan; khi triển khai trong môi trường sản xuất thực tế, độ dài modulus RSA cần nâng lên tối thiểu 2048 bit kèm cơ chế đệm RSA-OAEP.

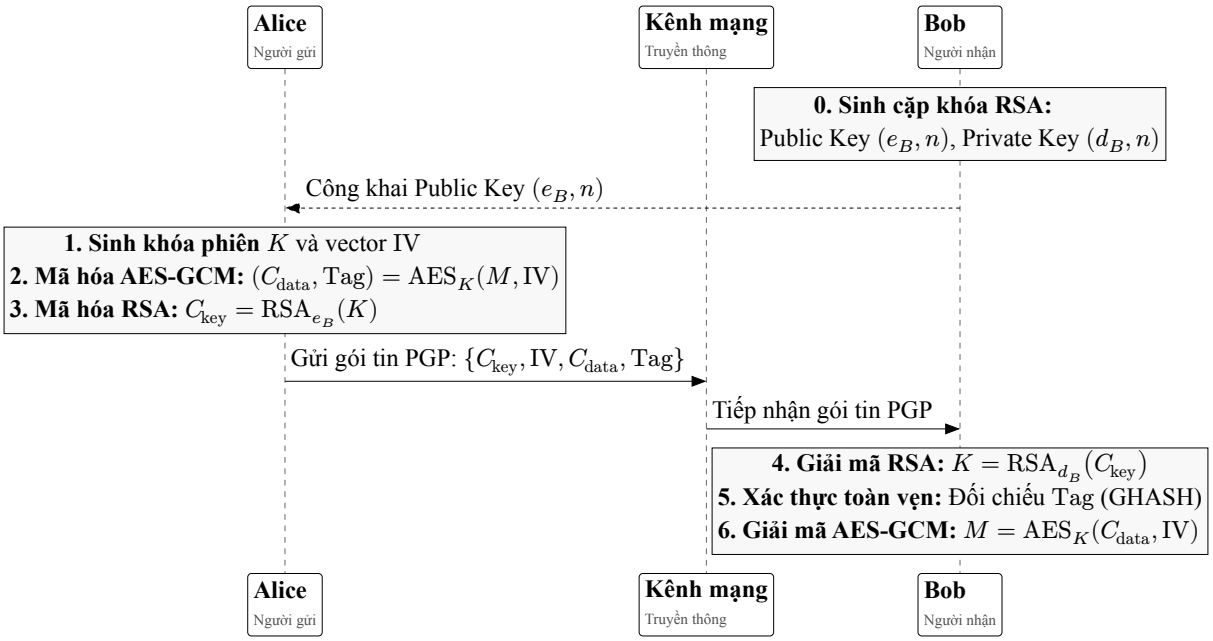
3. Phương pháp thực hiện

3.1. Quy trình mã hóa lai và phân vai thuật toán

Trong kiến trúc truyền thông giữa Alice (bên gửi) và Bob (bên nhận), mỗi lớp mật mã đảm nhiệm một vai trò độc lập: RSA giải quyết bài toán trao đổi khóa qua kênh mạng không tin cậy, còn AES-GCM bảo vệ nội dung thông điệp và chống sửa đổi dữ liệu. Quy trình năm giai đoạn được tóm tắt trong Bảng 4 và minh họa tuần tự tại Hình 2.

Bảng 4. Quy trình thực hiện mã hóa lai giữa Alice và Bob

Giai đoạn	Phía Alice (Người gửi)	Phía Bob (Người nhận)
GĐ 0: Khởi tạo khóa	Nhận khóa công khai (e, n) của Bob và đối chiếu dấu vân tay SHA-256.	Sinh cặp khóa RSA gồm khóa công khai (e, n) và khóa bí mật (d, n).
GĐ 1: Sinh khóa phiên	Dùng bộ sinh số ngẫu nhiên an toàn tạo khóa phiên K dài 128 bit (16 byte).	Mở cổng mạng chờ nhận kết nối từ phía gửi.
GĐ 2: Mã hóa hai lớp	• Dùng AES-GCM mã hóa M bằng K tạo bản mã dữ liệu, vector khởi tạo và thẻ xác thực. - Dùng RSA mã hóa K bằng (e, n) tạo bản mã khóa phiên.	Không thực hiện.
GĐ 3: Đóng gói & Gửi	Đóng gói bốn thành phần vào cấu trúc JSON kèm tiền tố 4 byte độ dài và gửi qua TCP.	Đọc 4 byte độ dài rồi nhận trọn vẹn phần thân gói tin vào bộ nhớ đệm.
GĐ 4: Giải mã & Kiểm tra	Chờ phản hồi trạng thái xử lý từ bên nhận.	• Dùng khóa bí mật d giải mã RSA khôi phục khóa phiên K'. - Tính lại thẻ xác thực bằng GHASH; nếu khớp mới giải mã ra bản rõ M'.



Hình 2. Sơ đồ tuần tự quy trình truyền thông mã hóa lai PGP giữa Alice và Bob

3.2. Cơ sở toán học và kích thước khóa của RSA

Hệ mật RSA sử dụng trong đề tài thực hiện phép tính số học trên số nguyên lớn theo bốn bước:

1. **Sinh số nguyên tố xác suất Miller–Rabin:** Hai số nguyên tố phân biệt p và q được kiểm tra bằng giải thuật Miller–Rabin [12] với $k = 20$ vòng lặp ngẫu nhiên, cho xác suất nhầm hợp số nhỏ hơn $\left(\frac{1}{4}\right)^{20} \approx 9.09 \times 10^{-13}$.
2. **Tính modulus n và hàm Euler:** Modulus hệ thống được tính theo Phương trình 2 và hàm phi Euler theo Phương trình 3:

$$n = p \cdot q \tag{2}$$

$$\varphi(n) = (p-1)(q-1) \tag{3}$$

3. **Chọn số mũ công khai Fermat:** Chọn cố định $e = 65537 = 2^{16} + 1$ vì biểu diễn nhị phân 10000000000000001_2 chỉ có hai bit 1, giúp phép bình phương và nhân liên tiếp chỉ cần 16 phép bình phương và 1 phép nhân theo mô-đun.
4. **Tìm khóa bí mật bằng giải thuật Euclid mở rộng:** Giải phương trình đồng dư ở Phương trình 4 để tìm d trong thời gian $O(\log(\varphi(n)))$:

$$e \cdot d \equiv 1 (\text{mod}\, \varphi(n)) \tag{4}$$

Cần phân biệt rạch ròi ý nghĩa số bit của từng đại lượng trong hệ thống:

- Với bản mô phỏng thuật toán trên giao diện web và kiểm thử nhanh, từng số nguyên tố p, q dài 128 bit nên modulus $n = p \cdot q$ dài 256 bit. Cấu hình này giúp các con số trung gian đủ gọn để quan sát trực tiếp từng bước biến đổi số học.
- Với mô hình truyền thông Socket qua mạng LAN, từng số nguyên tố p, q dài 256 bit nên modulus n dài 512 bit nhằm tăng biên độ an toàn cho khóa công khai truyền qua mạng mà vẫn bảo đảm thời gian sinh khóa nhanh.

- Trong cả hai cấu hình, khóa phiên AES gốc K luôn có kích thước cố định 128 bit ($K < 2^{128} < n$), còn bản mã khóa phiên C_{key} sau khi mã hóa RSA có kích thước bằng kích thước modulus n (256 bit hoặc 512 bit).

Quá trình mã hóa và giải mã RSA tại Phương trình 5 và Phương trình 6 thỏa mãn tính đúng đắn nhờ Định lý Euler:

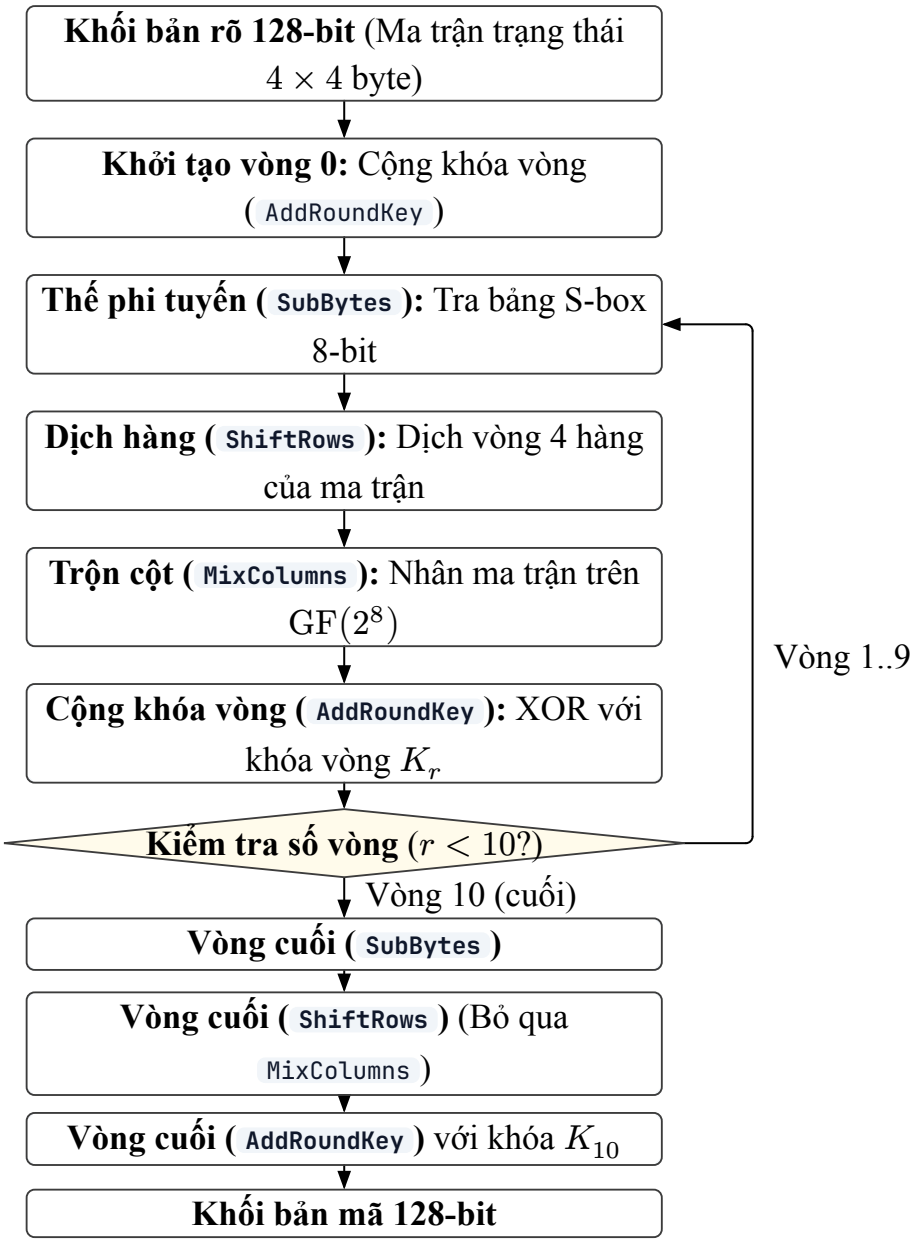
$$C_{\text{key}} = K^e \bmod n \tag{5}$$

$$K' = \left(C_{\text{key}}\right)^d \bmod n = K^{e \cdot d} \bmod n \tag{6}$$

Vì $e \cdot d = 1 + k \cdot \varphi(n)$, khi $\gcd(K, n) = 1$ thì $K^{\varphi(n)} \equiv 1 (\bmod n)$, nên $K' \equiv K \cdot 1^k \equiv K (\bmod n)$. Trường hợp hiếm gặp $\gcd(K, n) \neq 1$ (K chia hết cho p hoặc q), kết hợp Định lý Fermat nhỏ trên từng thừa số p, q và Định lý Số dư Trung Hoa (CRT) vẫn bảo đảm $K^{e \cdot d} \equiv K (\bmod n)$.

3.3. Lý thuyết và lưu đồ thuật toán mã hóa khối AES-128

AES là thuật toán mã hóa khối đối xứng hoạt động trên từng khối dữ liệu 128 bit (16 byte), được tổ chức thành ma trận trạng thái kích thước 4×4 byte. Với khóa phiên K dài 128 bit, dữ liệu đi qua bước cộng khóa khởi tạo và 10 vòng biến đổi theo cấu trúc mạng thay thế – hoán vị (SPN) như trình bày trong Hình 3.



Hình 3. Lưu đồ thuật toán mã hóa khối AES-128 theo cấu trúc mạng thay thế – hoán vị (SPN)

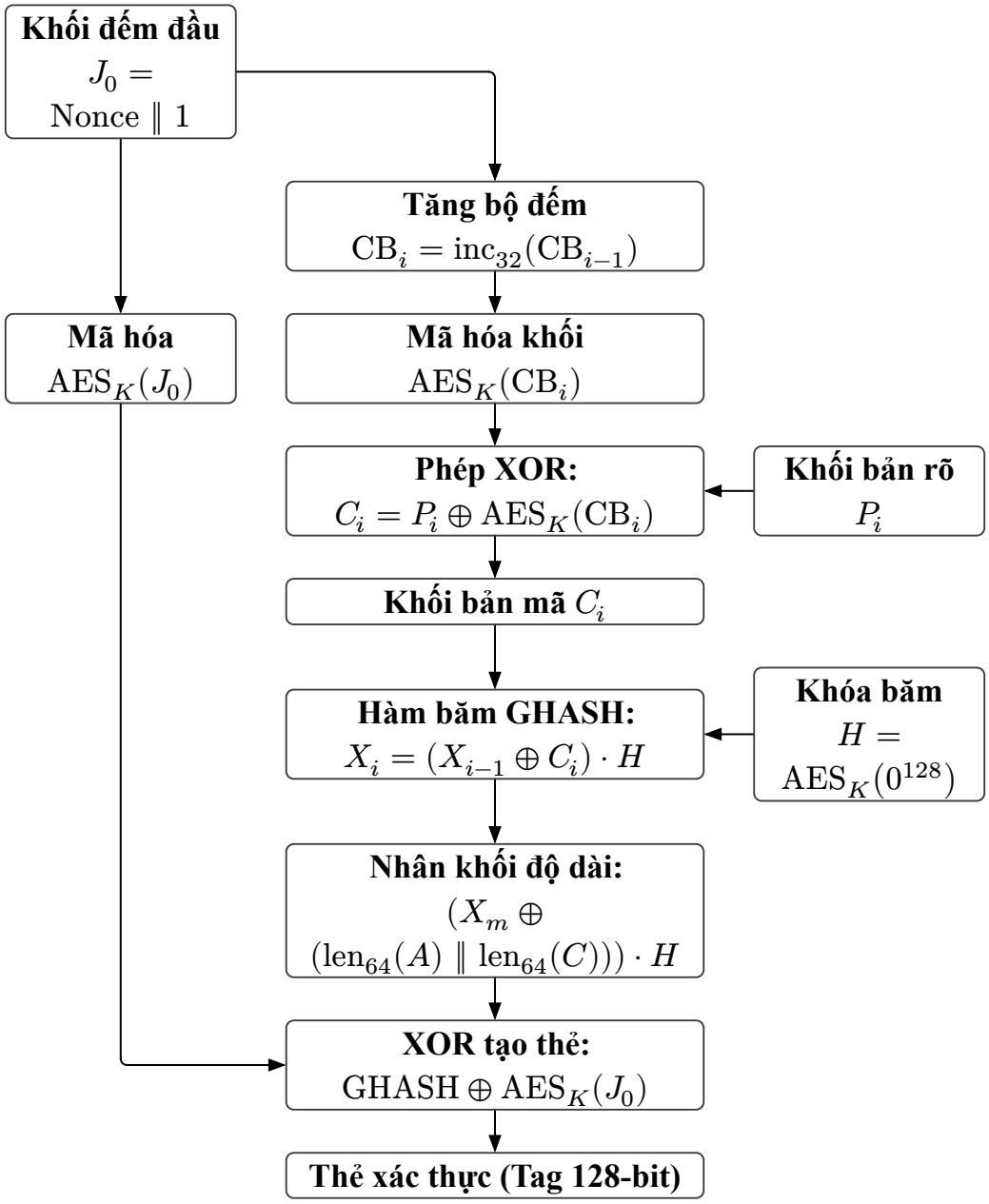
Mỗi vòng biến đổi của AES thực hiện bốn phép toán đại số cơ bản sau:

- **Thế phi tuyến (`SubBytes`):** Thay thế từng byte trong ma trận trạng thái bằng một byte tương ứng trong bảng thế S-box cố định nhằm tạo tính phi tuyến, chống lại phân tích mật mã tuyến tính và vi sai.
- **Dịch hàng (`ShiftRows`):** Dịch vòng sang trái từng hàng của ma trận trạng thái (hàng 0 giữ nguyên, hàng 1 dịch 1 byte, hàng 2 dịch 2 byte, hàng 3 dịch 3 byte) để khuếch tán dữ liệu theo chiều ngang.
- **Trộn cột (`MixColumns`):** Nhân từng cột của ma trận trạng thái với một đa thức cố định trên trường hữu hạn $\mathrm{GF}(2^8)$ nhằm trộn đều ảnh hưởng của từng byte trên toàn cột (phép toán này được bỏ qua ở vòng cuối thứ 10).

- **Cộng khóa vòng (`AddRoundKey`):** Thực hiện phép cộng loại trừ theo từng bit (XOR) giữa ma trận trạng thái hiện tại với khóa vòng K_r sinh ra từ thuật toán mở rộng khóa.

3.4. Chế độ mã hóa xác thực AES-GCM (AEAD)

AES-GCM thuộc họ mật mã xác thực có dữ liệu bổ sung AEAD (**Authenticated Encryption with Associated Data**), cho phép bảo vệ đồng thời tính bí mật của bản rõ và tính toàn vẹn của cả bản mã lẫn dữ liệu đi kèm (nếu có). Lưu đồ nguyên lý của AES-GCM theo chuẩn NIST SP 800-38D [10] được thể hiện trong Hình 4.



Hình 4. Lưu đồ nguyên lý mã hóa dòng CTR và xác thực toàn vẹn GHASH trong chế độ AES-GCM

Với vector khởi tạo Nonce dài 96 bit (12 byte), khối đếm ban đầu J_0 được tạo theo Phương trình 7 và các khối đếm mã hóa tiếp theo tăng dần 32 bit cuối theo Phương trình 8:

$$J_0 = \mathrm{CB}_0 = \mathrm{Nonce} \parallel 00000001_{16} \tag{7}$$

$$\mathrm{CB}_i = \mathrm{inc}_{32}(\mathrm{CB}_{i-1}) = \mathrm{Nonce} \parallel (i+1)_{32} \quad (i \geq 1) \tag{8}$$

Bản rõ M được chia thành các khối 128 bit $P_1, P_2, ..., P_m$ và mã hóa dòng theo chế độ bộ đếm (CTR) bằng Phương trình 9, loại bỏ nhu cầu đệm khối dữ liệu:

$$C_i = P_i \oplus \mathrm{AES}_K(\mathrm{CB}_i) \tag{9}$$

Song song với quá trình tạo bản mã, hàm băm GHASH tạo khóa băm $H = \mathrm{AES}_K(0^{128})$ và thực hiện phép nhân tích lũy trên trường Galois $\mathrm{GF}(2^{128})$ theo đa thức bất khả quy ở Phương trình 10:

$$f(x) = x^{128} + x^7 + x^2 + x + 1 \tag{10}$$

Nếu khối bản mã cuối C_m chưa đủ 128 bit, hệ thống đệm thêm các bit 0 về bên phải. Giá trị băm tích lũy qua từng khối là $X_i = (X_{i-1} \oplus C_i) \cdot H$ với $X_0 = 0^{128}$. Cuối cùng, khối độ dài 128 bit chứa độ dài tính theo bit của dữ liệu bổ sung A (rỗng trong hệ thống này) và bản mã C được nhân thêm để tạo thẻ xác thực 128-bit tại Phương trình 11:

$$\mathrm{Tag} = ((X_m \oplus (\mathrm{len}_{64}(A) \parallel \mathrm{len}_{64}(C))) \cdot H) \oplus \mathrm{AES}_K(J_0) \tag{11}$$

Nhờ tính khuếch tán trên trường $\mathrm{GF}(2^{128})$, chỉ cần 1 bit trong bản mã bị thay đổi trên đường truyền, thẻ xác thực tính lại tại bên nhận sẽ khác hoàn toàn với thẻ xác thực nhận được. Khi phát hiện sai lệch, bên nhận lập tức báo lỗi kiểm tra toàn vẹn và hủy gói tin thay vì giải mã ra dữ liệu sai.

3.5. Đóng gói khung truyền thông Socket TCP và xác thực dấu vân tay

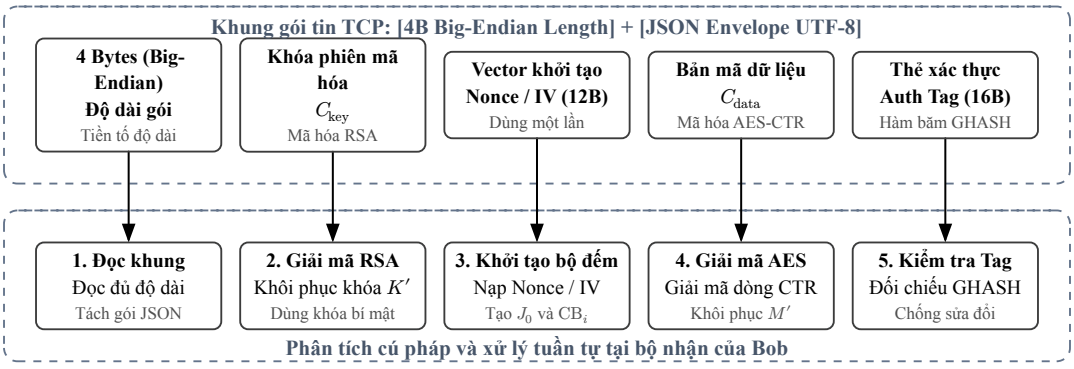
Giao thức TCP truyền dữ liệu theo luồng byte liên tục nên có thể xảy ra hiện tượng phân mảnh hoặc dính gói. Để phân tách ranh giới từng bức điện, bên gửi gắn tiền tố 4 byte số nguyên không dấu theo thứ tự byte mạng (Big-Endian) quy định kích thước chính xác của gói JSON đi kèm. Bốn thành phần mật mã trong gói tin được mô tả tại Bảng 5 và quy trình phân tích tại bên nhận thể hiện trong Hình 5.

Bảng 5. Cấu trúc bốn thành phần mật mã trong gói tin truyền qua mạng

Thành phần	Kích thước gốc	Biểu diễn truyền tải	Vai trò mật mã
Khóa phiên mã hóa	256 hoặc 512 bit	Số nguyên / Hex	Khóa phiên 128-bit đã mã hóa bằng khóa công khai của Bob qua RSA.
Vector khởi tạo	12 byte (96 bit)	Base64 / Hex	Giá trị ngẫu nhiên dùng một lần cho chế độ AES-GCM.
Bản mã dữ liệu	Bằng độ dài M	Base64 / Hex	Nội dung thông điệp M đã mã hóa đối xứng bằng AES-CTR.
Thẻ xác thực	16 byte (128 bit)	Base64 / Hex	Mã kiểm tra toàn vẹn sinh bởi hàm băm GHASH.

Để phòng ngừa tấn công giả mạo khóa công khai (tấn công kẻ đứng giữa), hệ thống tính dấu vân tay số dài 64 ký tự thập lục phân theo hàm băm SHA-256 ở Phương trình 12 để Alice và Bob đối chiếu qua kênh ngoài trước khi truyền tin:

$$\text{fingerprint}(\text{khóa công khai}) = \text{SHA256}(\text{RSA:} \parallel e \parallel : \parallel n) \tag{12}$$



Hình 5. Sơ đồ cấu trúc khung gói tin (Packet Framing) và luồng phân tích cú pháp tại bộ nhận của Bob

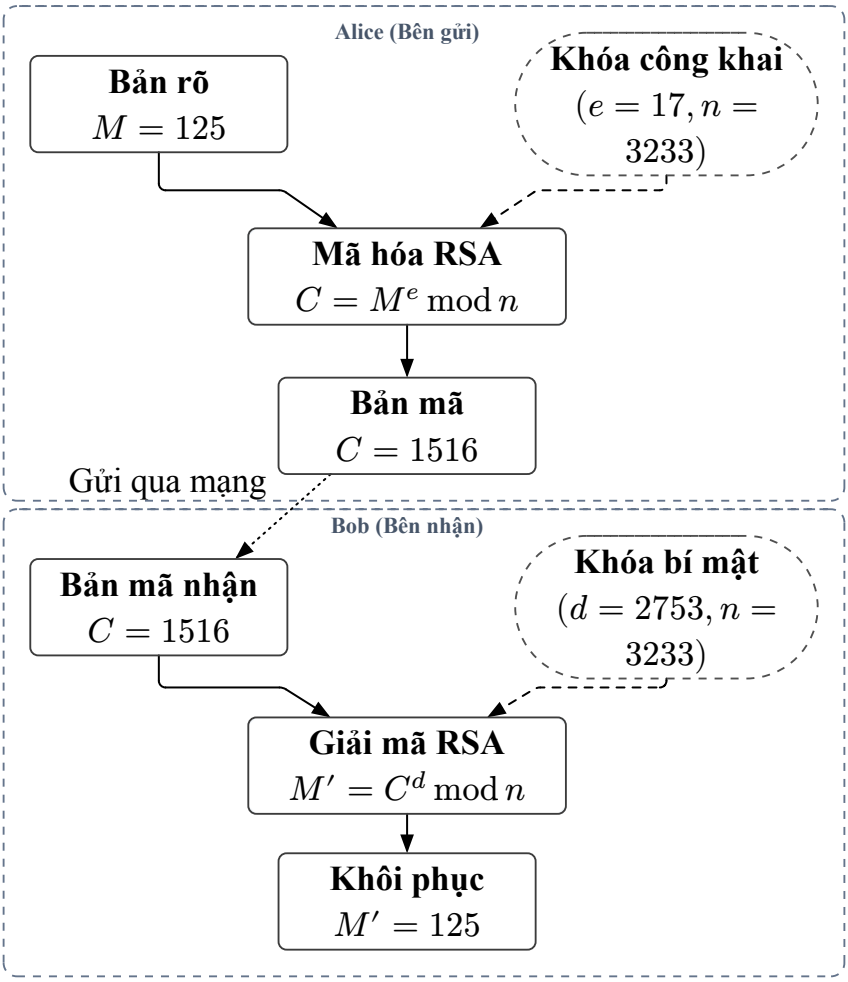
4. Thực nghiệm và kết quả

4.1. Ví dụ số học minh họa thuật toán RSA trên không gian nhỏ

Để kiểm chứng tính đúng đắn của thuật toán RSA tự cài đặt bằng phép tính thủ công, nhóm thực hiện thử nghiệm trên không gian số nhỏ với các tham số tổng hợp trong Bảng 6 và chu trình xử lý minh họa tại Hình 6.

Bảng 6. Tham số thực nghiệm tính toán số học trên không gian nhỏ của RSA

Bước tính toán	Công thức số học áp dụng	Giá trị kết quả
1. Chọn số nguyên tố	Chọn hai số nguyên tố p, q	$p = 61, \quad q = 53$
2. Tính modulus	$n = p \cdot q = 61 \times 53$	$n = 3233$
3. Tính hàm Euler	$\varphi(n) = (p-1)(q-1) = 60 \times 52$	$\varphi(n) = 3120$
4. Chọn khóa công khai	$\gcd(e, \varphi(n)) = 1, \quad 1 < e < \varphi(n)$	$e = 17$
5. Tìm khóa bí mật	$17 \cdot d \equiv 1 (\mathrm{mod}\, 3120)$ qua Euclid mở rộng	$d = 2753$ (vì $17 \times 2753 = 46801 = 15 \times 3120 + 1$)
6. Mã hóa và giải mã	Bản rõ $M = 125$ Mã hóa: $C = 125^{17} \bmod 3233$ Giải mã: $M' = 1516^{2753} \bmod 3233$	Bản mã $C = 1516$ Bản rõ khôi phục $M' = 125$ (trùng khớp)



Hình 6. Sơ đồ chu trình xử lý dữ liệu và khóa trong ví dụ số học RSA giữa Alice và Bob

4.2. Kết quả kiểm thử ba kịch bản an toàn

Nhóm triển khai chương trình thử nghiệm trên ngôn ngữ Python kết hợp thư viện mật mã PyCryptodome với khóa RSA 256-bit (hai số nguyên tố 128-bit) và khóa phiên AES 128-bit để đánh giá ba kịch bản: truyền thông hợp lệ, tấn công sửa đổi bản mã trên đường truyền và tấn công giải mã bằng khóa bí mật sai. Kết quả tóm tắt được trình bày trong Bảng 7.

Bảng 7. Tóm tắt kết quả thực nghiệm ba kịch bản kiểm thử

Kịch bản	Mô tả ngắn gọn	Kết quả
Kịch bản 1: Truyền thông hợp lệ	Bên nhận dùng đúng khóa bí mật của mình để giải mã gói tin từ bên gửi.	Đạt (PASSED)
Kịch bản 2: Sửa bit bản mã	Kẻ tấn công đảo 1 byte đầu tiên của bản mã dữ liệu trước khi gói tin tới bên nhận.	Đạt (PASSED)
Kịch bản 3: Dùng sai khóa bí mật	Bên thứ ba dùng một khóa bí mật khác để giải mã gói tin đánh cắp.	Đạt (PASSED)

Dưới đây là nhật ký chạy thực tế của từng kịch bản để kiểm chứng kết quả đạt yêu cầu:

- **Kịch bản 1 (Truyền thông hợp lệ):** Gói tin gồm khóa phiên mã hóa 256-bit, vector khởi tạo 12 byte, bản mã dữ liệu 81 byte và thẻ xác thực 16 byte được bên nhận giải mã chính xác 100%.

```
[ALICE] Thông điệp gốc: "Báo cáo tiến độ Nhóm 7: PGP Hybrid Encryption hoạt động chính xác!"
[ALICE] Mã hóa hoàn tất:
  + Khóa phiên mã hóa (RSA)     : 0x3db2e725bbdfa6ccfdc2ad8aa86796631e...
  + Vector khởi tạo (AES-GCM)   : 41878945b25c12f1f69b5057 (12 bytes)
  + Bản mã dữ liệu (AES)        : bb80889066cd1281a1282c00ce2a3b0e... (81 bytes)
  + Thẻ xác thực (Tag)          : 1139bb3c42099c5b3984cfd8f0235632 (16 bytes)

TEST CASE 01: Bob dùng đúng Private Key của mình
[BOB] Giải mã thành công: "Báo cáo tiến độ Nhóm 7: PGP Hybrid Encryption hoạt động chính xác!"
>>> KẾT QUẢ: [PASSED] Khôi phục chính xác 100% dữ liệu.
```

- **Kịch bản 2 (Tấn công sửa bit bản mã):** Khi 1 byte của bản mã dữ liệu bị đảo bit trên đường truyền, giá trị băm GHASH tính lại không khớp với thẻ xác thực nên bên nhận hủy gói tin.

```
TEST CASE 02: Kẻ tấn công sửa đổi 1 byte trong Ciphertext
[BOB] CẢNH BÁO AN TOÀN: Phát hiện dữ liệu bị can thiệp!
      Chi tiết lỗi: Kiểm tra thẻ xác thực thất bại (Tag không khớp)
>>> KẾT QUẢ: [PASSED] Chặn thành công tấn công sửa đổi bản mã.
```

- **Kịch bản 3 (Dùng sai khóa bí mật):** Dùng khóa bí mật khác để giải mã khiến khóa phiên khôi phục vượt quá 128 bit hoặc sai lệch, bị hệ thống từ chối ngay lập tức.

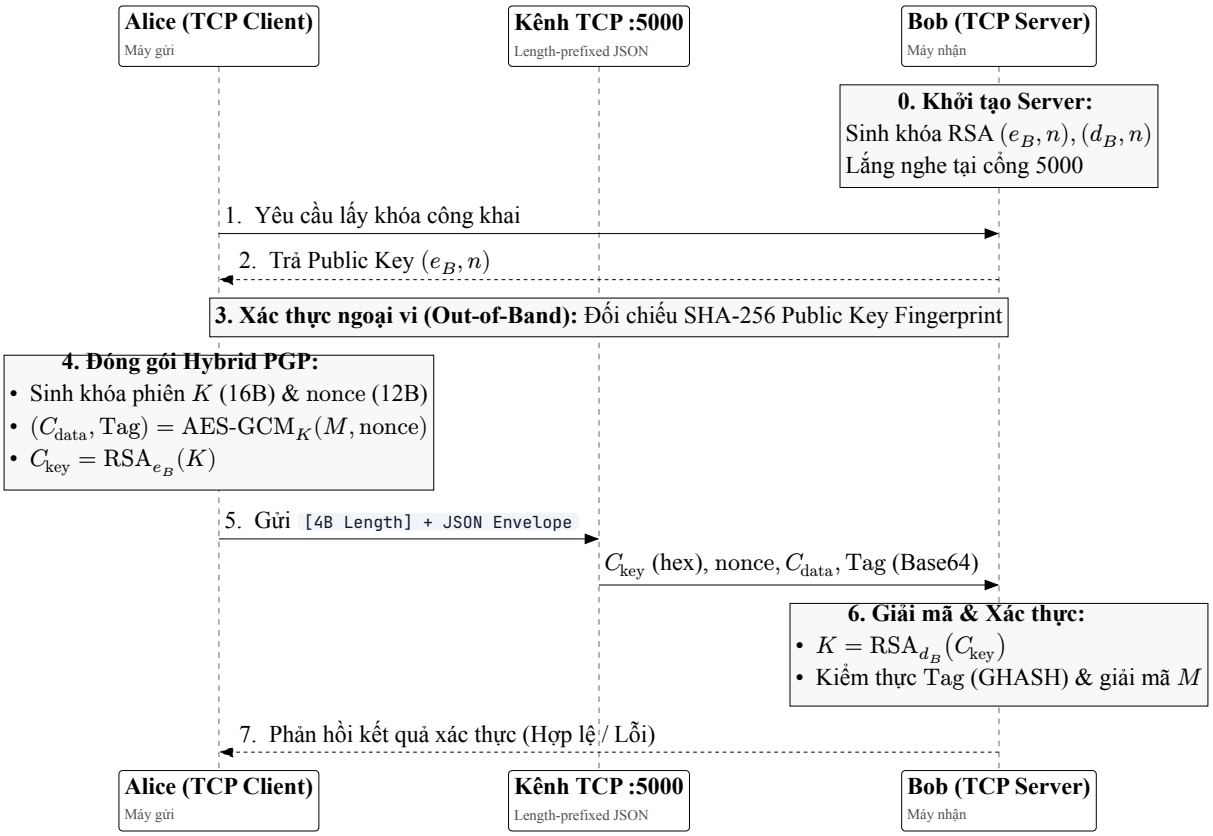
```
TEST CASE 03: Giải mã bằng Private Key không hợp lệ (Khóa của Eve)
[EVE] Thất bại: Giải mã RSA ra Session Key sai -> Từ chối xác thực.
      Chi tiết lỗi: Khóa phiên giải mã vượt quá 128-bit (sai khóa hoặc bị can thiệp)
>>> KẾT QUẢ: [PASSED] Từ chối truy cập trái phép khi sai khóa.
```

4.3. Thực nghiệm truyền thông Socket TCP giữa hai máy mạng LAN

Để kiểm chứng hoạt động trên môi trường mạng thực tế, nhóm triển khai thử nghiệm giữa hai máy tính kết nối trong cùng mạng Wi-Fi cục bộ theo mô hình Khách – Chủ (Client–Server) qua giao thức TCP ở cổng 5000, như minh họa trong Hình 7.

Tại máy nhận (đóng vai trò Server), chương trình tự động sinh cặp khóa RSA 512-bit (từ hai số nguyên tố 256-bit), hiển thị địa chỉ IP cục bộ cùng dấu vân tay SHA-256 của khóa công khai và mở cổng lắng nghe kết nối. Tại máy gửi (đóng vai trò Client), người dùng nhập địa chỉ IP của máy nhận để kết nối, tự động tải về khóa công khai và đối chiếu dấu vân tay SHA-256 nhằm xác nhận đúng người nhận. Sau khi người gửi

nhập thông điệp và chọn chế độ truyền (gửi hợp lệ, giả lập sửa bản mã hoặc giả lập sai khóa), gói tin được đóng khung kèm tiền tố 4 byte độ dài và truyền qua mạng LAN. Quá trình truyền nhận và giải mã hoàn tất với độ trễ dưới 15 ms; các gói tin bị sửa đổi đều bị máy nhận phát hiện và từ chối xử lý.



Hình 7. Sơ đồ tuần tự mô phỏng mã hóa lai qua Socket TCP giữa máy Alice và máy Bob

4.4. Giao diện mô phỏng trực quan trên nền web

Nhằm hỗ trợ quan sát trực tiếp quá trình mã hóa và giải mã, nhóm xây dựng ứng dụng mô phỏng tương tác trên nền web (sử dụng thư viện NiceGUI) gồm hai chế độ màn hình:

1. **Chế độ mô phỏng luồng tích hợp:** Trình bày bố cục ba cột gồm bên gửi (Alice), kênh truyền mạng và bên nhận (Bob). Người dùng có thể thực hiện từng bước hoặc chạy tuần tự toàn bộ quy trình, theo dõi sơ đồ trạng thái đổi màu theo thời gian thực và kích hoạt giả lập sửa bit trực tiếp trên gói tin trước khi bên nhận giải mã.
2. **Chế độ chi tiết từng khối thuật toán:** Hiển thị năm khối tính toán liên hoàn từ trên xuống dưới gồm sinh khóa RSA, mã hóa dữ liệu AES-GCM, mã hóa khóa phiên bằng RSA, giải mã khóa phiên bằng RSA và giải mã AES-GCM khôi phục bản rõ. Khóa phiên 128-bit thực tế được truyền xuyên suốt qua các khối giúp người học kiểm tra sự nhất quán giữa lý thuyết số học và dữ liệu chạy thật.

5. Kết luận và hướng phát triển

Đề tài đã hoàn thành việc nghiên cứu và mô phỏng cơ chế mã hóa lai kết hợp RSA và AES-128-GCM theo mô hình OpenPGP. Về thuật toán, nhóm nắm vững cấu trúc mạng thay thế – hoán vị của AES, cơ chế xác thực GHASH trên trường $\mathrm{GF}(2^{128})$ và tự cài đặt trọn vẹn RSA số học thuần bằng Python (Miller–Rabin, Euclid mở rộng).

Về triển khai thực tế, hệ thống đã giải quyết được bài toán phân mảnh luồng byte TCP bằng khung tiền tố độ dài và đối chiếu dấu vân tay khóa công khai qua SHA-256 trên mạng cục bộ. Kết quả kiểm thử ba kịch bản và giao diện mô phỏng xác nhận dữ liệu hợp lệ được khôi phục chính xác, còn các gói tin bị sửa bit hoặc giải mã sai khóa đều bị phát hiện và loại bỏ.

Trong thời gian tới, hệ thống có thể mở rộng theo ba hướng chính:

- **Bổ sung chữ ký số:** Kết hợp SHA-256 và khóa bí mật của bên gửi để bảo đảm thêm tính xác thực nguồn gốc và chống chối bỏ.
- **Nâng cấp chuẩn an toàn RSA:** Tăng độ dài khóa lên tối thiểu 2048 bit kèm cơ chế đệm ngẫu nhiên RSA-OAEP theo RFC 8017.
- **Đo đạc hiệu năng với dữ liệu lớn:** Thử nghiệm truyền tệp đa phương tiện dung lượng lớn qua Socket TCP để đánh giá định lượng tốc độ giữa mã hóa lai và mã hóa thuần bất đối xứng.

TÀI LIỆU THAM KHẢO

[1] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 7th a.b. Pearson, 2017.

[2] National Institute of Standards and Technology, "Advanced Encryption Standard (AES)", Federal Information Processing Standards Publication 197, 2001.

[3] R. L. Rivest, A. Shamir, và L. Adleman, "A Method for Obtaining Digital Signatures and Public-Key Cryptosystems", *Communications of the ACM*, tập 21, số 2, tr 120–126, 1978.

[4] J. Callas, L. Donnerhacke, H. Finney, D. Shaw, và R. Thayer, "OpenPGP Message Format", RFC 4880, 2007.

[5] P. Wouters và D. Huigens, "OpenPGP", RFC 9580, 2024.

[6] X. Zhou và X. Tang, "Research and implementation of RSA algorithm for encryption and decryption", trong *Proceedings of 2011 6th International Forum on Strategic Technology (IFOST)*, IEEE, 2011, tr 1118–1121.

[7] K. Moriarty, B. Kaliski, J. Jonsson, và A. Rusch, "PKCS #1: RSA Cryptography Specifications Version 2.2", RFC 8017, 2016.

[8] D. Coppersmith, "Small Solutions to Polynomial Equations, and Low Exponent RSA Attacks", *Journal of Cryptology*, tập 10, số 4, tr 233–260, 1997.

[9] D. A. McGrew và J. Viega, "The Security and Performance of the Galois/Counter Mode (GCM) of Operation", trong *Fast Software Encryption (FSE 2004)*, trong Lecture Notes in Computer Science, vol. 3017. Springer, 2004, tr 343–355.

[10] M. Dworkin, "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC", NIST Special Publication 800–38D, 2007.

[11] W. Diffie và M. E. Hellman, "New Directions in Cryptography", *IEEE Transactions on Information Theory*, tập 22, số 6, tr 644–654, 1976.

[12] M. O. Rabin, "Probabilistic algorithm for testing primality", *Journal of Number Theory*, tập 12, số 1, tr 128–138, 1980.

PHỤ LỤC

Phụ lục A: Cấu trúc mã nguồn và hướng dẫn chạy chương trình

Hệ thống phần mềm được tổ chức phân tầng với lõi thuật toán đặt tại thư mục `algorithms/` và tái sử dụng cho ứng dụng web NiceGUI, kịch bản kiểm thử CLI và mô hình Socket LAN:

- `algorithms/euclid.py`: Cài đặt giải thuật Euclid mở rộng (`extended_gcd`) và tìm nghịch đảo mô-đun (`mod_inverse`).
- `algorithms/rsa.py`: Cài đặt kiểm tra nguyên tố Miller–Rabin (`is_prime`, mặc định `k = 20`), sinh số nguyên tố (`generate_prime`), sinh cặp khóa (`generate_rsa_keys`), mã hóa (`rsa_encrypt`) và giải mã (`rsa_decrypt`).
- `algorithms/aes_gcm.py`: Cài đặt mã hóa (`aes_gcm_encrypt`) và giải mã có kiểm thực (`aes_gcm_decrypt`) dựa trên PyCryptodome.
- `algorithms/hybrid_pgp.py`: Đóng gói (`alice_encrypt_package`) và giải mã (`bob_decrypt_package`) luồng dữ liệu lai.
- `PGP_RSA_AES_Socket_3_files/`: Gồm `pgp_crypto.py`, `server_bob.py` (TCP Server, khóa RSA 512-bit) và `client_alice.py` (TCP Client).
- `src/hybrid_demo.py`: Kịch bản dòng lệnh chạy 3 kịch bản kiểm thử an toàn.
- `src/gui_nicegui.py`: Ứng dụng web mô phỏng tương tác trên nền NiceGUI (chạy tại cổng `8080`).

Hướng dẫn cài đặt thư viện và chạy thử nghiệm từ thư mục gốc dự án:

```bash
pip install -r requirements.txt
python3 src/hybrid_demo.py          # Chạy 3 kịch bản kiểm thử CLI
python3 src/gui_nicegui.py          # Khởi động Web Simulator tại http://localhost:8080
python3 PGP_RSA_AES_Socket_3_files/server_bob.py        # Chạy trên máy nhận (Bob)
python3 PGP_RSA_AES_Socket_3_files/client_alice.py <IP> # Chạy trên máy gửi (Alice)
```

Phụ lục B: Cài đặt giải thuật Euclid mở rộng và RSA số học

Mã nguồn mô-đun `algorithms/euclid.py`:

```python
def extended_gcd(a, b):
    if a == 0:
        return b, 0, 1
    gcd, x1, y1 = extended_gcd(b % a, a)
    x = y1 - (b // a) * x1
    y = x1
    return gcd, x, y

def mod_inverse(e, phi):
    gcd, x, _ = extended_gcd(e, phi)
    if gcd != 1:
```

```
        raise ValueError(f"Không tồn tại nghịch đảo modulo: gcd({e}, {phi}) = {gcd} != 1")
    return (x % phi + phi) % phi
```

Mã nguồn mô-đun `algorithms/rsa.py` :

```python
import random
from .euclid import extended_gcd, mod_inverse

def is_prime(n, k=20):
    if n < 2:
        return False
    if n in (2, 3):
        return True
    if n % 2 == 0:
        return False

    r, d = 0, n - 1
    while d % 2 == 0:
        r += 1
        d //= 2

    rng = random.SystemRandom()
    for _ in range(k):
        a = rng.randrange(2, n - 1)
        x = pow(a, d, n)
        if x == 1 or x == n - 1:
            continue
        for _ in range(r - 1):
            x = pow(x, 2, n)
            if x == n - 1:
                break
        else:
            return False
    return True

def generate_prime(bits=128):
    rng = random.SystemRandom()
    while True:
        candidate = rng.getrandbits(bits) | (1 << (bits - 1)) | 1
        if is_prime(candidate):
            return candidate

def generate_rsa_keys(p=None, q=None, e=65537, bits=256):
    if p is None and q is None:
        prime_bits = max(bits // 2, 72)
        while True:
            p = generate_prime(prime_bits)
            q = generate_prime(prime_bits)
```

```
            if p != q and extended_gcd(e, (p - 1) * (q - 1))[0] == 1:
                break
    elif p is None or q is None:
        raise ValueError("Phải cung cấp đồng thời cả p và q, hoặc để trống cả hai để sinh ngẫu nhiên.")
    else:
        if p == q or not is_prime(p) or not is_prime(q):
            raise ValueError("p và q phải là hai số nguyên tố phân biệt.")
        if extended_gcd(e, (p - 1) * (q - 1))[0] != 1:
            raise ValueError("e và phi(n) phải nguyên tố cùng nhau.")

    n = p * q
    phi = (p - 1) * (q - 1)
    d = mod_inverse(e, phi)
    return (e, n), (d, n)

def rsa_encrypt(public_key, plaintext_num):
    e, n = public_key
    if not (0 <= plaintext_num < n):
        raise ValueError(f"Dữ liệu ({plaintext_num}) phải nằm trong khoảng [0, {n - 1}]")
    return pow(plaintext_num, e, n)

def rsa_decrypt(private_key, ciphertext_num):
    d, n = private_key
    if not (0 <= ciphertext_num < n):
        raise ValueError(f"Bản mã ({ciphertext_num}) phải nằm trong khoảng [0, {n - 1}]")
    return pow(ciphertext_num, d, n)
```

Phụ lục C: Cài đặt AES-GCM và bộ đóng gói mã hóa lai

Mã nguồn mô-đun `algorithms/aes_gcm.py` và `algorithms/hybrid_pgp.py`:

Python

```python
import os
from Cryptodome.Cipher import AES
from .rsa import rsa_encrypt, rsa_decrypt

def aes_gcm_encrypt(session_key, plaintext_bytes):
    nonce = os.urandom(12)
    cipher = AES.new(session_key, AES.MODE_GCM, nonce=nonce)
    ciphertext, auth_tag = cipher.encrypt_and_digest(plaintext_bytes)
    return ciphertext, nonce, auth_tag

def aes_gcm_decrypt(session_key, nonce_bytes, ciphertext_bytes, auth_tag_bytes):
    if len(nonce_bytes) != 12:
        raise ValueError(f"Nonce AES-GCM chuẩn phải dài 12 bytes (nhận được {len(nonce_bytes)} bytes).")
    cipher = AES.new(session_key, AES.MODE_GCM, nonce=nonce_bytes)
    return cipher.decrypt_and_verify(ciphertext_bytes, auth_tag_bytes)
```

```
16
17 # --- Đóng gói PGP Hybrid ---
18 def alice_encrypt_package(plaintext_str, bob_pub_key):
19     session_key = os.urandom(16)  # Khóa phiên AES-128 (16 bytes = 128 bits)
20     ciphertext, nonce, auth_tag = aes_gcm_encrypt(session_key,
       plaintext_str.encode("utf-8"))
21     key_int = int.from_bytes(session_key, byteorder="big")
22     enc_session_key = rsa_encrypt(bob_pub_key, key_int)
23     package = {
24         "enc_session_key": enc_session_key,
25         "nonce": nonce,
26         "ciphertext": ciphertext,
27         "auth_tag": auth_tag,
28     }
29     return package, session_key
30
31 def bob_decrypt_package(package, bob_priv_key):
32     recovered_key_int = rsa_decrypt(bob_priv_key, package["enc_session_key"])
33     try:
34         recovered_session_key = recovered_key_int.to_bytes(16, byteorder="big")
35     except OverflowError as exc:
36         raise ValueError("Khóa phiên giải mã từ RSA vượt quá 128-bit (sai khóa hoặc
       C_key bị can thiệp).") from exc
37     plaintext_bytes = aes_gcm_decrypt(
38         session_key=recovered_session_key,
39         nonce_bytes=package["nonce"],
40         ciphertext_bytes=package["ciphertext"],
41         auth_tag_bytes=package["auth_tag"],
42     )
43     return plaintext_bytes.decode("utf-8")
```

Phụ lục D: Cài đặt truyền thông Socket TCP giữa 2 máy mạng LAN

Cơ chế đóng khung 4 byte độ dài giải quyết phân mảnh byte stream trong `PGP_RSA_AES_Socket_3_files/server_bob.py` :

```
1  import struct, json                                                    Python
2
3  MAX_PACKET = 4 * 1024 * 1024
4
5  def recv_exact(connection, count):
6      chunks = bytearray()
7      while len(chunks) < count:
8          chunk = connection.recv(min(count - len(chunks), 65536))
9          if not chunk:
10             raise ConnectionError("Bên gửi đã đóng kết nối trước khi gửi đủ dữ
               liệu.")
11         chunks.extend(chunk)
```

```
12      return bytes(chunks)
13
14  def send_json(connection, payload_dict):
15      data = json.dumps(payload_dict, separators=(",", ":")).encode("utf-8")
16      connection.sendall(struct.pack("!I", len(data)) + data)
17
18  def receive_json(connection):
19      packet_length = struct.unpack("!I", recv_exact(connection, 4))[0]
20      if not 0 < packet_length <= MAX_PACKET:
21          raise ValueError("Kích thước gói tin không hợp lệ.")
22      packet = recv_exact(connection, packet_length)
23      envelope = json.loads(packet.decode("utf-8"))
24      if not isinstance(envelope, dict):
25          raise ValueError("Gói tin phải là một JSON object.")
26      return envelope
```